Implementation & Simulation

The "easy" problems where regionals are lost

Learning goals

- Recognise implementation-heavy problems and budget time for them
- Design before typing: state, data layout, helper functions
- Parse awkward input robustly (`getline`, `stringstream`, recursive descent)
- Use symmetry tricks (rotation, direction arrays) to cut code in half
- Simulate huge step counts with cycle detection

Why this lecture exists

- Every regional has 1–3 problems that are "just do what it says"
- Typical damage: 40 min of coding + 2 WAs = **80 min of penalty**
- Top teams solve them in one go because they **design first**
- The skill is engineering: small functions, no duplicated branches, invariants checked

Spotting an implementation problem

- Long statement with many rules, small constraints
- Words like *simulate*, *process the commands*, *game*, *format*
- The algorithm is obvious; the **details** are not
- Output format is intricate (tables, alignment, special cases)

> Ask: "what is the state, and what is one transition?"

Design on paper: 5-minute plan

1. **State**: which variables fully describe the situation?
2. **Transitions**: one function per rule
3. **Representation**: arrays over maps; integers over strings
4. **Corner cases** from the statement, listed explicitly
5. **Output**: write the exact format down

Only then sit at the keyboard.

Direction arrays

```
// N, E, S, W — turning right is d = (d + 1) % 4
const int dr[4] = {-1, 0, 1, 0};
const int dc[4] = {0, 1, 0, -1};

auto inside = [&](int r, int c) {
    return 0 <= r && r < R && 0 <= c && c < C;
};
for (int d = 0; d < 4; d++) {
    int nr = r + dr[d], nc = c + dc[d];
    if (inside(nr, nc) && g[nr][nc] != '#') { /* ... */ }
}
```

- 8 neighbours / knight moves: just longer arrays
- Never write four copy-pasted `if` blocks

Symmetry: write one case, rotate

- "Move tiles left / up / right / down" (the 2048 game)
- Implement **only "left"**; rotate the board so every move becomes left

```
dir:   left  up  right  down
CCW rotations before:   0     1     2     3
CCW rotations after:    0     3     2     1
```

- Same trick: mirror strings, transpose matrices, reflect coordinates

2048: one row to the left

```
void slideLeft(array<int, 4>& row) {
    vector<int> v, out;
    for (int x : row) if (x) v.push_back(x);        // compress
    for (size_t i = 0; i < v.size(); i++) {
        if (i + 1 < v.size() && v[i] == v[i + 1]) {
            out.push_back(2 * v[i]); i++;           // merge once
        } else out.push_back(v[i]);
    }
    for (int i = 0; i < 4; i++)
        row[i] = i < (int)out.size() ? out[i] : 0;
}
```

- Compress → merge → pad: three clean phases, no index juggling

Parsing toolkit

Input shape	Tool
Tokens separated by whitespace	`cin >> x`
Whole lines with spaces	`getline(cin, line)`
Split a line into tokens	`stringstream ss(line); while (ss >> w)`
Fixed formats like `12:34`	`scanf("%d:%d", &h, &m)`
Nested / recursive syntax	recursive descent parser

- Mixing `cin >>` and `getline`: call `cin.ignore()` or `getline` once to eat the newline

Example: Codeforces 1B *Spreadsheets*

- Convert `BC23` ⇄ `R23C55`
- Column letters are **bijective base 26** (no zero digit)

```
string toLetters(long long c) {  // 1→A, 26→Z, 27→AA
    string s;
    while (c > 0) { c--; s += char('A' + c % 26); c /= 26; }
    reverse(s.begin(), s.end());
    return s;
}
```

- Classic trap: `R1C1`-looking strings like `RC23`? Check "digit after R" *and* a later `C`

Recursive descent: grammar first

```
expr   := term   (('+' | '-') term)*
term   := factor (('*' | '/') factor)*
factor := '-' factor | '(' expr ')' | number
```

- One function per grammar rule
- Precedence comes from the nesting, not from special cases
- Same pattern parses chemical formulas, nested lists, boolean expressions, tree descriptions like `(a(b)(c))`

Recursive descent: code

```
ll expr() {
    ll v = term();
    while (peek() == '+' || peek() == '-') {
        char op = s[pos++]; ll r = term();
        v = op == '+' ? v + r : v - r;
    }
    return v;
}
ll factor() {
    char c = peek();
    if (c == '-') { pos++; return -factor(); }
    if (c == '(') { pos++; ll v = expr(); peek(); pos++; return v; }
    ll v = 0;
    while (pos < s.size() && isdigit(s[pos])) v = v*10 + (s[pos++]-'0');
    return v;
}
```

Simulation with huge step counts

- "Simulate K = 10^{18} steps" with a small state space
- The sequence of states is **eventually periodic** (pigeonhole)

```
states:  s0 → s1 → s2 → s3 → s4 → s5 → s3 → ...
                          └──── cycle len 3 ────┘
```

- Record first time each state is seen; at the repeat:
  - `len = t - first[state]`
  - jump: simulate only `(K - t) % len` more steps

Cycle detection: code core

```
vector<long long> seen(4LL * R * C, -1);
for (long long t = 0; t < K; t++) {
    int s = id(r, c, d);
    if (seen[s] != -1) {
        long long rem = (K - t) % (t - seen[s]);
        while (rem--) step();       // finish inside the cycle
        break;
    }
    seen[s] = t;
    step();
}
```

- State must be **complete**: position *and* direction (and any counters)
- If the state is big, hash it into a `map<State, long long>`

Event simulation

- Customers, elevators, processes: events at times, processed in order
- Use a `priority_queue` of `(time, type, id)` with a **defined tie order**
- Statement says "if two events happen simultaneously …" → encode in the tuple

```
priority_queue<tuple<ll,int,int>, vector<tuple<ll,int,int>>,
               greater<>> pq;     // min-heap by time, then type
```

Clean code under pressure

- Name things: `inside()`, `step()`, `applyRule3()`
- Keep global state small; pass grids by reference
- No magic numbers: `const int N = 4;`
- One responsibility per loop
- Assert invariants while testing: `assert(0 <= r && r < R);`

Testing an implementation

- Samples first, then **your own tiny cases** for each rule in the statement
- Make a checklist from the statement: one test per sentence with "if", "unless", "except"
- Compare against a slow, dumb version when the fast one is clever (e.g. cycle jumping vs. plain loop)

Complexity check

Task	Cost
Grid simulation, cycle detection	O(R·C·4) states
Recursive descent	O(length), recursion depth = nesting
Event simulation	O(E log E)
2048 move	O(16)

- Watch recursion depth: 10^6 nested parentheses → stack overflow; use an explicit stack

Common pitfalls

- Mixing `cin >>` and `getline` (leftover newline)
- Off-by-one in 1-indexed statements vs 0-indexed arrays
- Incomplete state in cycle detection → wrong period
- `int` overflow in step counters (`K ≤ 10^18`)
- Tie rules in simultaneous events ignored
- Printing `-0.00` or wrong rounding

Real regional flavour

- Kattis `2048` — the 2048 move above, exactly
- UVa 10267 *Graphical Editor* — command interpreter with flood fill
- UVa 10196 *Check The Check* — chess rules, direction arrays
- UVa 10142 *Australian Voting* — multi-round elimination rules
- Kattis `robotturtles` — parsing a board + BFS on states (bridges to Lecture 4)

Practice problems

- Codeforces 1B — Spreadsheets
- Codeforces 4C — Registration System
- Kattis `2048`
- UVa 727 — Equation (infix → postfix)
- UVa 10267 — Graphical Editor
- UVa 10196 — Check The Check
- UVa 10142 — Australian Voting
- Kattis `robotturtles` — Robot Turtles

Summary

- Design state and transitions on paper before typing
- Direction arrays and rotation symmetry remove duplicated code
- Parsing: pick the tool by input shape; recursive descent for grammars
- Huge simulations: detect the cycle, jump with modular arithmetic
- Next: advanced greedy — when "obvious" needs a proof